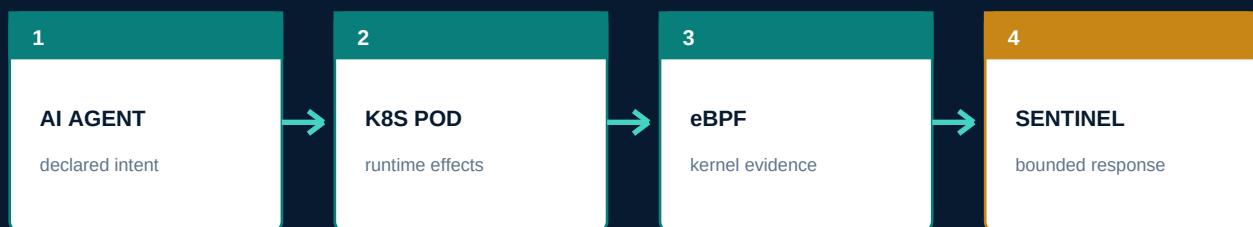


AGENT SENTINEL

Intent-Aware Runtime Detection and Adaptive Containment of Compromised AI Agents in Kubernetes Using eBPF

SHIVAM KUMAR | RESEARCH WHITE PAPER 0.1 | 25 AUGUST 2026



Authorize the action. Verify its effects. Contain deviations.

Architecture proposal + executable research MVP. Quantitative comparative claims remain to be evaluated.

Abstract

Tool-using AI agents join probabilistic decision-making with credentials, network access, files, and system tools. A prompt injection, compromised dependency, malicious tool response, or stolen runtime identity can therefore become a real process, file, network, or control-plane action. Application-layer authorization can inspect the agent's proposed tool call, but it does not prove that the resulting container performed only the effects implied by that call. Conversely, existing runtime security systems observe kernel effects but lack the task context needed to distinguish expected agent behavior from compromise.

Agent Sentinel proposes an intent-aware runtime security architecture for Kubernetes. A task or tool action produces a constrained behavioral contract; Tetragon supplies Kubernetes-enriched eBPF events; a correlator compares process, file, network, and Kubernetes API effects with the active contract; and a response controller escalates from observation to capability restriction and Pod quarantine. This white paper defines the threat model, architecture, prototype, experimental method, and claim boundaries. The accompanying v0.1 implementation validates contracts, parses normalized and Tetragon events, produces explainable decisions, emits reversible Kubernetes containment plans, and includes a monitor-first eBPF/Kubernetes lab. Comparative security and performance results are future work.

1. Problem Statement

Large-language-model agents increasingly invoke browsers, code interpreters, databases, source-code platforms, and infrastructure APIs. The agent often consumes untrusted text in the same context used to choose a privileged action. InjecAgent and AgentDojo provide empirical evidence that indirect prompt injection can redirect tool-using agents and motivate realistic security evaluation [1, 2]. Recent work studies capability boundaries, provenance, and tool-call interception [3, 4, 5].

These defenses address an essential decision boundary, but the tool call is not the last security boundary. Consider an approved action named `summarize_document`. Its implementation may import a compromised package that reads a service-account token and opens an outbound socket. The high-level call was expected; its hidden system effects were not. A kernel observer can see those effects but does not know that summarization required local file reads and no network access.

The missing link is runtime authorization conditioned on high-level intent:

`authorized action -> expected system effects -> observed kernel effects -> bounded response`

The research objective is to determine whether this correlation improves detection and containment without unacceptable loss of legitimate agent utility.

2. Design Principles

2.1 Effects, not reasoning traces

Agent Sentinel does not require chain-of-thought inspection or assume that a model can reliably judge its own compromise. It evaluates observable system effects. This makes the mechanism compatible with different models and orchestration frameworks.

2.2 Positive, task-conditioned authorization

The same operation can be legitimate for one action and suspicious for another. An HTTPS connection is expected during literature search but not during an explicitly offline summarization task. Contracts therefore describe positive permissions per intent, with a small set of task-independent hard denials.

2.3 Monitor first, enforce deliberately

Runtime enforcement can interrupt useful work or race with the protected operation. Tetragon documents both return-value override and signal-based response, including the fact that killing a process does not necessarily prevent an operation already in progress [7]. Agent Sentinel separates evidence, decision, and enforcement and defaults to response-plan output.

2.4 Explain every decision

The MVP uses deterministic findings and a transparent risk ledger. A decision identifies the violated intent clause, event, score, and proposed control. Learned anomaly detection may be added only with explainability, drift handling, and a static-invariant fallback.

2.5 Containment must be bounded and reversible

The controller's implemented mutation is a Pod label. A pre-installed deny-all NetworkPolicy selects that label. Automatic deletion and credential rotation are intentionally not implemented because they have broader operational impact.

3. Threat Model

The protected assets are workload credentials, mounted data, tool authority, Kubernetes resources, downstream services, and the integrity of agent actions. Entry paths include indirect prompt injection, malicious tools or MCP servers, compromised packages or images, credential theft, and code execution inside an agent container.

The adversary may control the agent process and its children but does not control the node kernel, Tetragon sensor, Kubernetes API server, CNI, contract issuer, or Agent Sentinel controller. Node-root, kernel, and sensor compromise are outside the v0.1 scope. This boundary is important because eBPF monitoring itself is privileged and therefore a high-value target [6].

Covered behaviors include unapproved process execution, reads of designated Secret and token paths, connections outside the active intent, and unauthorized Kubernetes operations. Agent Sentinel cannot infer malicious semantics when every system effect is allowed, recover a Secret after it was read, or inspect encrypted application content at the kernel layer.

4. System Architecture

The architecture has four planes.

4.1 Intent plane

An `AgentBehaviorContract` selects a Kubernetes workload and defines one or more intents. Each intent contains allowed executable globs, file-path globs, destination hostnames or CIDRs, ports, protocols, and Kubernetes `verb:resource` operations. Hard-deny process and file patterns override an intent.

The v0.1 prototype reads the intent from a Pod label. The research design replaces that label with a signed, short-lived action lease bound to Pod UID, process lineage, capability set, expiry, and nonce. The lease should be issued by a trusted tool gateway before execution and should not be writable by the agent workload.

4.2 Evidence plane

Tetragon observes process lifecycle, system calls, file and network activity, and associates events with Kubernetes identity [6]. Agent Sentinel normalizes core process-exec, connect, kprobe, and LSM event families. A future adapter will ingest Kubernetes audit events for control-plane operations.

LSM hooks are valuable for security decisions because they sit at common permission points rather than requiring a rule for every possible read system call. Availability depends on the Linux kernel's BPF LSM configuration [11].

4.3 Correlation plane

For every selected workload event, the detector resolves the active intent and checks:

- whether an executable is hard-denied or absent from the intent;
- whether a file is sensitive or outside the intent;
- whether destination identity, port, and protocol are permitted;
- whether the Kubernetes API operation is allowed;
- whether repeated deviations raise cumulative workload risk.

Expected events reduce accumulated risk. Unknown intents and events outside the contract raise explicit findings. The current risk function is intentionally simple and is a baseline, not a claim of optimal anomaly detection.

4.4 Response plane

Scores map to allow, restrict, or contain. Restriction represents inline denial, rate limiting, or an approval gate appropriate to the capability. Containment labels the Pod `sentinel.shivam.dev/state=quarantined`; an empty-ingress/egress NetworkPolicy then isolates it. Network policy enforcement requires a compatible CNI and operates at L3/L4 [8].

For Pod-bound projected service-account tokens, Kubernetes validation can invalidate the token when the bound Pod is deleted; projected tokens also expire and rotate [9]. This does not rotate unrelated application or cloud credentials, which require separate provider-specific response.

5. Prototype Implementation

The accompanying Apache-2.0 repository provides an installable Python package and Kubernetes lab.

The contract loader rejects wrong API versions, missing selectors, duplicate intent identifiers, invalid CIDRs, and inconsistent thresholds. Network suffix matching preserves DNS label boundaries, so `notarxiv.org` does not satisfy `arxiv.org`. The detector produces a finding code, reason, severity, and risk score for each deviation.

The sensor adapter accepts stable normalized JSONL for reproducible experiments and best-effort parsing of Tetragon JSON exports. The responder constructs `kubectl label` as an argument vector and executes it only when `--apply-response` is present. Shell interpolation is not used. The responder service account has `get`, `list`, and `patch` on Pods; it cannot delete Pods or read Secrets.

The lab includes a demo agent that runs non-root with RuntimeDefault seccomp, no Linux capabilities, a read-only root filesystem, bounded resources, and no mounted service-account token. A Tetragon policy observes protected Secret paths in monitor mode. An opt-in sibling policy uses BPF LSM override to return `EACCES`; it is not installed by default.

Nineteen unit tests currently cover contract validation, host/suffix/CIDR matching, expected behavior, hard denials, risk accumulation, event parsing, and response-plan construction. These tests establish software behavior, not real-cluster security effectiveness.

6. Research Questions and Hypotheses

The evaluation has four research questions:

- Does intent improve detection precision and recall relative to static runtime rules?
- Does staged enforcement reduce completed unauthorized effects and containment latency?
- How much benign task utility is lost, especially for legitimate novel behavior?
- What event-processing and end-to-end workload overhead does the design add?

The primary hypothesis is that intent plus runtime evidence improves precision at matched recall. A second hypothesis is that inline denial prevents more unauthorized effects than signal-only or post-event quarantine. A third predicts that per-task contracts overblock less than a global allowlist. The target of less than 5% median task-latency overhead is explicitly prospective.

7. Experimental Plan

Four configurations should be compared: Kubernetes hardening alone; static Falco/Tetragon rules; an intent/tool boundary without kernel correlation; and the hybrid design. Each runs the same paired benign and adversarial scenarios in randomized order.

Workloads should span a literature agent, repository agent, and operations agent. Attack families should include indirect prompt injection, malicious tool output, compromised dependencies, credential abuse, and living-off-the-land behavior. Rare-but-valid tasks are necessary to measure false positives, not merely obvious attacks.

Ground truth must come from independent effect sinks: a controlled canary file read, request received by a test endpoint, or audit-confirmed forbidden API operation. A detector alert alone is not attack success. Primary metrics are precision, recall, F1, attack success, unauthorized-effect completion, benign task success, decision latency, verified containment latency, event loss, CPU, memory, and p95 task latency.

Publish raw counts and confidence intervals per attack family. Use paired scenarios, bootstrap intervals, McNemar's test for paired binary outcomes where appropriate, and non-parametric latency comparisons. Pre-register primary endpoints and retain pilot data outside the final test set.

8. Limitations and Open Problems

Authorization authenticity. A writable Pod label is not a secure lease. Lease issuance and binding are the highest-priority architectural gap.

Destination identity. Socket telemetry naturally exposes IP addresses, while human contracts often name domains. DNS correlation must resist workload-controlled claims, caching ambiguity, proxies, and content-delivery networks.

Semantic gap. A permitted interpreter can implement many behaviors. File and socket effects narrow the gap but cannot prove that an allowed API request was semantically safe.

TOCTOU and enforcement coverage. Hook placement and kernel configuration determine whether an operation can be denied before effect. Policies must be tested on every supported node image.

Drift. Agent images, dependencies, tasks, and endpoint addresses change. Monitor-first rollout and contract versioning are necessary to prevent silent overblocking.

Sensor integrity and availability. A privileged runtime sensor expands the trusted computing base. Missing telemetry must not be mistaken for benign inactivity.

Multi-agent causality. Delegation across agents and services makes a single-Pod intent insufficient. Future work needs cross-workload causal identity without turning one compromised agent into a trusted issuer for another.

9. Responsible Use

The project is a defensive research prototype. Synthetic traces use reserved IP space and no real credentials. Monitor mode is the default. Researchers should isolate experiments, use canary secrets, avoid public attack infrastructure, and obtain authorization before observing workloads. Raw production telemetry may contain sensitive paths, arguments, or destinations and should not be published.

Claims must distinguish code tests, lab measurements, and hypotheses. Comparative claims should not be made until the versioned methodology and raw results are available for reproduction.

10. Conclusion

AI-agent security requires controls at more than one layer. Tool-boundary authorization captures task context; eBPF runtime evidence captures actual system effects; Kubernetes provides workload identity and containment primitives. Agent Sentinel combines these layers around a testable idea: authorize a bounded action, verify its runtime consequences, and constrain deviations before they become broader incidents.

The v0.1 repository turns that idea into an auditable starting point with explicit limitations. The next research milestone is not a larger dashboard or a more complex anomaly model. It is a controlled, reproducible experiment that shows when intent correlation adds security, when it harms utility, and which effects can be prevented in time.

References

1. Q. Zhan, Z. Liang, Z. Ying, and D. Kang. [InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents](#). 2024.
2. E. Debenedetti et al. [AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents](#). 2024.
3. F. Alpay and T. Alpay. [AgentSecBench: Measuring Prompt Injection, Privacy Leakage, and Tool-Use Integrity in LLM Agents](#). 2026 preprint.
4. W. Zhao et al. [ClawGuard: A Runtime Security Framework for Tool-Augmented LLM Agents Against Indirect Prompt Injection](#). 2026 preprint.
5. S. Weng et al. [ARGUS: Defending LLM Agents Against Context-Aware Prompt Injection](#). 2026 preprint.
6. Cilium Tetragon. [Overview](#) and [Threat Model](#). Accessed 25 August 2026.
7. Cilium Tetragon. [Enforcement](#) and [TracingPolicy](#). Accessed 25 August 2026.
8. Kubernetes. [Network Policies](#). Accessed 25 August 2026.
9. Kubernetes. [Service Accounts](#). Accessed 25 August 2026.
10. Falco. [Default Rules](#) and [Condition Syntax](#). Accessed 25 August 2026.
11. Linux kernel documentation. [LSM BPF Programs](#). Accessed 25 August 2026.